Rapport d'Audit de Sécurité Mobile

Cible et Itération : C:\Users\elsam\Bureau\Projet_MCP\owasp.sat.agoat_v1.0_202
Date d'exécution : 2026-09-19 18:57 Généré par : Agent MCP (audit_android)

Rapport de Test d'Intrusion — Application Android `owasp.sat.agoat` (AndroGoat)

Audit exhaustif — Analyse statique multi-vulnérabilités + PoC dynamique

Auditeur : Elsa-Marie — Pentest Mobile Senior **Date de l'audit :** 19 septembre 2026 **Cible :** `owasp.sat.agoat` (AndroGoat) — version 1.0 (versionCode 1) **Environnement :** Émulateur Android (ADB `emulator-5554`), toolchain `audit_android` (ADB + Jadx + Frida 16.3.3 via TCP) **Méthodologie :** Analyse statique (décompilation Jadx, revue manuelle du manifeste et du code source Java/Kotlin décompilé) + validation dynamique (instrumentation Frida)

---

1. Résumé exécutif

Cet audit exhaustif de l'application volontairement vulnérable AndroGoat (OWASP) a permis d'identifier **8 catégories de vulnérabilités distinctes**, couvrant l'ensemble du spectre du Mobile Application Security Testing Guide (MASTG) de l'OWASP : contrôle d'accès défaillant sur composants exportés, injection SQL (locale et via ContentProvider), injection de commande OS, configuration WebView dangereuse, secrets codés en dur, stockage non sécurisé de données sensibles, et contournement de la détection de root (validé dynamiquement par instrumentation Frida).

L'application `android:debuggable="true"` et `android:allowBackup="true"` en configuration par défaut aggrave l'impact de chacune de ces failles en facilitant leur exploitation par un attaquant local disposant d'un accès physique ou d'ADB.

Niveau de risque global : Critique. Plusieurs vulnérabilités (injection SQL sur ContentProvider exporté, injection de commande OS, WebView à accès fichier universel) permettent une exfiltration de données ou une exécution de code arbitraire sans nécessiter de privilèges root, et sont exploitables par une application tierce malveillante installée sur le même appareil.

---

2. Identification de la cible

Élément	Valeur

Deux méthodes assurent la vérification :

- `isRooted()` : parcourt une liste statique de chemins connus (`/system/bin/su`, `Superuser.apk`, binaires SuperSU, Xposed, etc.) via `File.exists()`.
- `isRooted1()` : exécute `su ls /data/data/` via `Runtime.exec()` et considère l'appareil rooté si aucune `IOException` n'est levée.

Faiblesse : vérification purement côté client, sans attestation d'intégrité (Play Integrity API), sans validation serveur, sans code natif obfusqué.

Preuve de contournement dynamique (validée, PID 6626) :

```
Java.perform(function () {
    var RootDetectionActivity = Java.use('owasp.sat.agoat.RootDetection
    RootDetectionActivity.isRooted.implementation = function () {
        console.log('[BYPASS] isRooted() intercepted -> forcing return
        return false;
    };
    RootDetectionActivity.isRooted1.implementation = function () {
        console.log('[BYPASS] isRooted1() intercepted -> forcing return
        return false;
    };
    console.log('[+] Hooks installes sur RootDetectionActivity');
});
```

Résultat confirmé par l'outil d'instrumentation : application spawnée et script injecté avec succès (PID 6626).

4.2 ContentProvider exporté vulnérable à l'injection SQL

Localisation : `owasp.sat.agoat.ContentProviderActivity` (déclaré comme `<provider>` exporté, authority `owasp.sat.agoat.provider.userpinsprovider`)

Le provider expose une base SQLite (`androgoat_userpins.db`, table `user_pins`) contenant des identifiants en clair (insérés en dur au premier lancement : `AndroGoat/AndroGoat`, `root/toor`, `Admin/Admin`). Les méthodes `delete()` et `update()` construisent la clause `WHERE` par **concaténation directe du segment d'URI** :

```
String where = "id = " + id + (!TextUtils.isEmpty(selection) ? " AND ("
count = sQLiteDatabase.delete(TABLE, where, selectionArgs);
```

Comme le provider est `exported="true"` sans permission de lecture/écriture déclarée, **toute application tierce installée sur l'appareil peut interroger, modifier ou supprimer les données via `ContentResolver`**, et injecter du SQL arbitraire via l'`id` de l'URI (ex. `content://owasp.sat.agoat.provider.userpinsprovider/user_pins/1 OR 1=1--`).

4.3 Injection SQL locale

Localisation : `owasp.sat.agoat.SQLinjectionActivity`

```
String qry = "SELECT * FROM users WHERE username='" + ((Object) $userna
```

```
...
Cursor qryResult = sQLiteDatabase.rawQuery(qry, null);
```

Aucune requête paramétrée (`? + selectionArgs`) n'est utilisée : une entrée telle que `' OR '1'='1` retourne l'intégralité de la table `users`. De plus, le résultat de la requête — **y compris les mots de passe** — est journalisé en clair via `Log.e("QueryResult", ...)` et `Log.v("Query", qry)`, exposant les données sensibles dans Logcat, accessible à toute application disposant de la permission `READ_LOGS` sur les versions d'Android concernées ou via `adb logcat`.

4.4 Injection de commande OS

Localisation :
`owasp.sat.agoat.InputValidationsOSCMDInjectionMain2Activity`

```
String ip1 = "ping " + ((Object) $ip.getText());
Process p = Runtime.getRuntime().exec(ip1);
```

L'entrée utilisateur est concaténée directement dans la commande shell sans aucune validation ni échappement. Bien qu'`exec(String)` sur Android ne passe pas par un shell interprétant `;`/`&&` nativement (contrairement à `exec("sh -c ...")`), l'absence totale de validation reste une faiblesse de conception majeure et rend l'application vulnérable dès qu'un shell interpréteur est introduit dans la chaîne (schéma classique de progression vers un command injection complet).

4.5 WebView à configuration dangereuse

Localisation : `owasp.sat.agoat.InputValidationsWebViewURLActivity`

```
webViewSettings.setJavaScriptEnabled(true);
webViewSettings.setAllowFileAccess(true);
webViewSettings.setAllowContentAccess(true);
webViewSettings.setAllowFileAccessFromFileURLs(true);
webViewSettings.setAllowUniversalAccessFromFileURLs(true);
String url = $urlEditText.getText().toString();
$webView.loadUrl(url);
```

Cette combinaison de paramètres est le pire scénario possible pour une WebView Android :

- **JavaScript activé + URL chargée directement depuis une saisie utilisateur/Intent** = vecteur XSS/injection.
- **`setAllowUniversalAccessFromFileURLs(true)`** couplé à **`setAllowFileAccessFromFileURLs(true)`** permet à une page `file://` malveillante de lire des ressources cross-origin, y compris potentiellement d'autres fichiers locaux de l'application (vol de données, contournement de la Same-Origin Policy).

Aucune liste blanche de domaines, aucun `WebViewClient` personnalisé avec validation d'URL, aucune désactivation de l'accès fichier n'est mise en œuvre.

4.6 Secrets codés en dur

`HardCodeActivity` :

```
private final String promoCode = "NEW2019";
```

`AIChatActivity` :

```
private final String openAIApiKey = "sk-abcdef1234567890abcdef123456789
```

Toute chaîne codée en dur dans le bytecode Dalvik/ART est trivialement récupérable par décompilation (comme démontré ici même via Jadx), sans nécessiter le moindre accès root ni instrumentation dynamique. Une clé d'API légitime codée de cette façon serait immédiatement exploitable par quiconque extrait l'APK (marché public ou privé, sideloading, etc.).

4.7 Stockage non sécurisé d'identifiants

Localisation : `owasp.sat.agoat.InsecureStorageSharedPrefs`

```
SharedPreferences sharedPreference = this$0.getSharedPreferences("users
SharedPreferences.Editor editor = sharedPreference.edit();
editor.putString(ContentProviderActivity.USERNAME, $username.getText().
editor.putString("password", $password.getText().toString());
editor.commit();
```

Le mode `0` (`MODE_PRIVATE`) protège certes le fichier des autres applications sur un appareil non rooté, mais **le mot de passe est stocké en clair, sans aucun chiffrement** (ni Android Keystore, ni `EncryptedSharedPreferences` de Jetpack Security). Sur un appareil rooté — ou via une sauvegarde ADB permise par `allowBackup="true"` — ces identifiants sont directement lisibles en texte clair dans `/data/data/owasp.sat.agoat/shared_prefs/users.xml`.

4.8 Contrôle d'accès défaillant sur composants exportés

`AccessControl1ViewActivity` (accessible via le deep link exporté `androgoat://vulnapp`) déclenche directement :

```
Intent serviceIntent = new Intent(this$0, (Class<?>) DownloadInvoiceSer
this$0.startService(serviceIntent);
```

`DownloadInvoiceService` est lui-même déclaré `exported="true"` dans le manifeste, **sans permission associée**. Conséquence : n'importe quelle application tierce peut invoquer directement ce service via :

```
adb shell am startservice -n owasp.sat.agoat/.DownloadInvoiceService
```

ou par un `Intent` explicite depuis une app malveillante, **sans passer par l'interface utilisateur ni par un quelconque contrôle métier**, déclenchant une action applicative (téléchargement de fichier) hors du flux prévu — un exemple représentatif de contournement de contrôle d'accès par composant exporté non protégé (`ShowDataReceiver`, également exporté sans permission, présente le même schéma de risque).

---

5. Recommandations de remédiation

Vulnérabilité	Remédiation
Détection de root	Combiner détection client et attestation serveur (Play Integrity API) ; déplacer la logique en natif obfusqué ; réagir activement (blocage) plutôt que informativement
ContentProvider exporté	Positionner `exported="false"` si aucun accès externe n'est requis ; sinon exiger une `android:permission` signature ; utiliser systématiquement des requêtes paramétrées (`selectionArgs`), jamais de concaténation
Injection SQL locale	Utiliser exclusivement des requêtes paramétrées (`rawQuery(sql, String[] args)` avec `?`) ; ne jamais logger de données sensibles (mots de passe, tokens)
Injection de commande OS	Éviter `Runtime.exec()` sur des entrées utilisateur ; utiliser des API Java natives (`InetAddress.isReachable()`) plutôt que d'invoquer un binaire externe ; valider/whitelister strictement le format attendu (IP)
WebView	Désactiver `setAllowFileAccessFromFileURLs` et `setAllowUniversalAccessFromFileURLs` ; restreindre la navigation à une liste blanche de domaines via un `WebViewClient` personnalisé ; ne jamais charger une URL directement issue d'une saisie utilisateur sans validation
Secrets codés en dur	Ne jamais embarquer de clé API dans le code client ; utiliser un backend proxy pour les appels nécessitant un secret ; si stockage local indispensable, utiliser Android Keystore
Stockage non sécurisé	Utiliser `EncryptedSharedPreferences` (Jetpack Security) ou chiffrement via Android Keystore ; désactiver `allowBackup` pour les données sensibles
Composants exportés	Passer `exported="false"` par défaut ; n'exporter que les composants strictement nécessaires, avec permission `signature` ou vérification de l'appelant (`getCallingPackage()`)
Configuration globale	Désactiver `android:debuggable` et `android:allowBackup` dans les builds de production

---

Rapport généré dans le cadre d'un test d'intrusion autorisé sur l'application de démonstration OWASP AndroGoat, à des fins éducatives et de validation d'outillage de sécurité mobile.